

TP: Money Laundering Analysis
75.74 Sistemas Distribuidos I
Primer cuatrimestre del 2026
Grupo 10

Nombre y Apellido	Padrón	Mail
Escandar Obadía, Germán Gonzalo	105250	gescandar@fi.uba.ar
Morseletto, Bruno Gabriel	104087	bmorseletto@fi.uba.ar
Alonso Schneider, Micaela Jazmin	109551	malonso@fi.uba.ar

Índice

Índice.....	2
Introducción.....	3
Escenarios.....	3
Vista Lógica.....	3
D.A.G.....	3
Vista de desarrollo.....	4
Diagrama de paquetes.....	4
Vista de proceso.....	5
Diagramas de actividad.....	5
Escenario 1.....	5
Escenario 2.....	7
Escenario 3.....	8
Escenario 4.....	10
Escenario 5.....	11
Diagrama de secuencia.....	12
Vista física.....	13
Diagrama de robustez.....	13
Diagrama de despliegue.....	13
Tareas a ejecutar.....	14
División de los integrantes.....	15
Decisiones tomadas.....	16
Tolerancia a fallos.....	18
Node Manager.....	18
Write-Ahead-Log.....	19
Chaos Monkey.....	20
Gateway.....	20
Referencias.....	21

Introducción

El presente informe propone el diseño de un sistema distribuido para analizar registros de transacciones realizadas entre diferentes cuentas bancarias con el objetivo de encontrar anomalías que podrían indicar que hubo un lavado de activos.

Escenarios

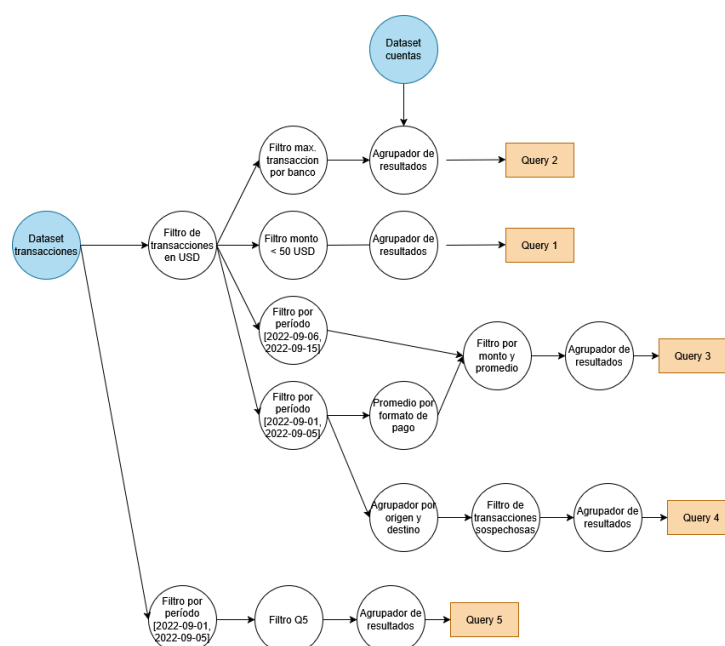
El análisis de los datos consiste en poder resolver las siguientes consultas:

1. Cuenta de origen, cuenta de destino y monto para transacciones USD menores a 50.
2. Nombre de banco, cuenta de origen y monto de la máxima transacción USD de cada banco.
3. Cuenta de origen y monto de transacciones USD en el período [2022-09-06, 2022-09-15] con monto menor a 1 centésimo del promedio encontrado para el mismo formato de pago en el período [2022-09-01, 2022-09-05].
4. Cuentas que cumplan con el patrón scatter-gather con una sola cuenta de separación, para cuentas que hayan realizado y cuya cuenta de origen haya realizado transferencias USD hacia entre 5 cuentas distintas dentro del período [2022-09-01, 2022-09-05].
5. Cantidad de transacciones del período [2022-09-01, 2022-09-05] con formato de pago "Wire" o "ACH" cuyo monto convertido a USD sea menor a 1.

Vista Lógica

D.A.G.

Para observar el workflow completo del procesamiento de los datos, contamos con el siguiente diagrama:



Podemos observar en color celeste las diferentes fuentes de datos (transacciones y cuentas) que el cliente provee. Luego, para resolver cada query, los datos pasan por diferentes filtros, agrupadores y convertidores hasta llegar al nodo final (Agrupador de resultados). Además, cabe mencionar que los datos correspondientes a las cuentas solo se utiliza para la query 2. Luego, se tiene que en algunos casos filtros cuyos resultados son de utilidad para varias queries, como lo es el filtro de transacciones en USD por ejemplo. Por último, los indicadores de que flujo corresponde a que query esta a modo ilustrativo, pues en el sistema implementado los agrupadores de resultados le envían los resultados finales al Gateway.

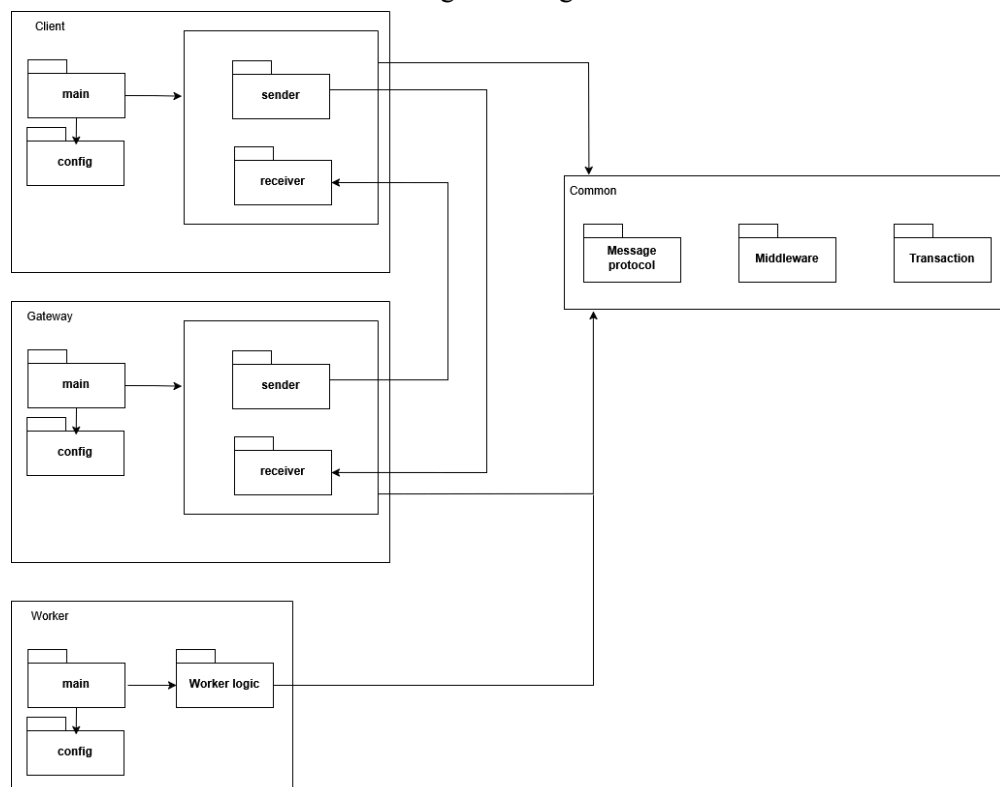
En cuanto al *Filtro Q5*, aquí se realizan diferentes chequeos y procesamiento: se convierte el monto a USD con los valores de cotización adecuados, se descartan las transacciones con formato de pago “Wire” o “ACH” y se descartan las transacciones cuyo monto supera 1 USD.

Vista de desarrollo

Diagrama de paquetes

A continuación se observan los diagramas correspondientes a la estructura tentativa del presente trabajo.

Por un lado, se tendrá un paquete de Common, el cual contará con el protocolo de mensajes y el Middleware del sistema. Luego, el resto del sistema será compuesto por los componentes de clientes, gateway y los diferentes workers. A modo de simplificación, se representó a los diferentes filtros, convertidores, agregadores y joins en un sólo módulo “Worker”, pues estos tendrán una similar estructura donde la única diferencia será la lógica de negocio dentro de los mismos.

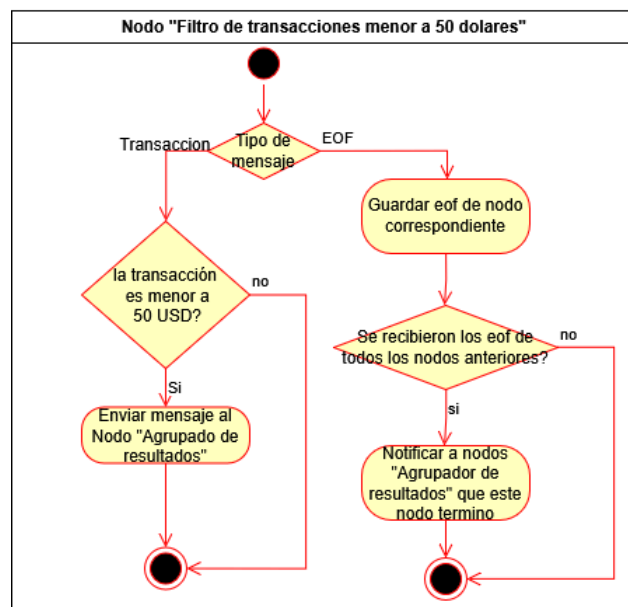
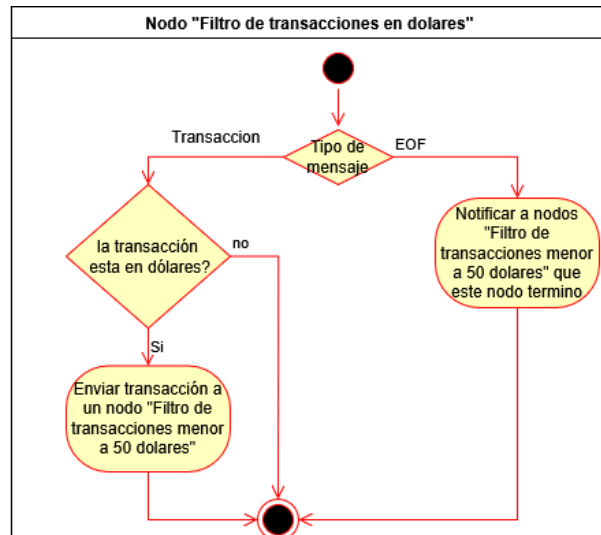


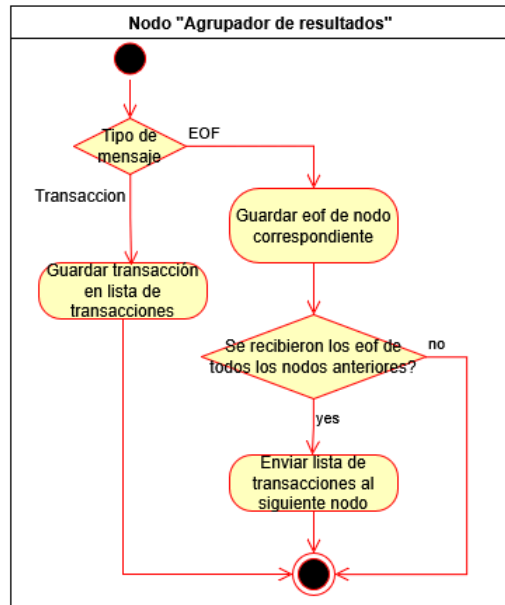
Vista de proceso

Diagramas de actividad

Los Diagramas de actividad muestran el flujo de acciones que se van tomando dentro del sistema sin tener en cuenta de manera muy detallada la implementación interna de cada componente.

Escenario 1

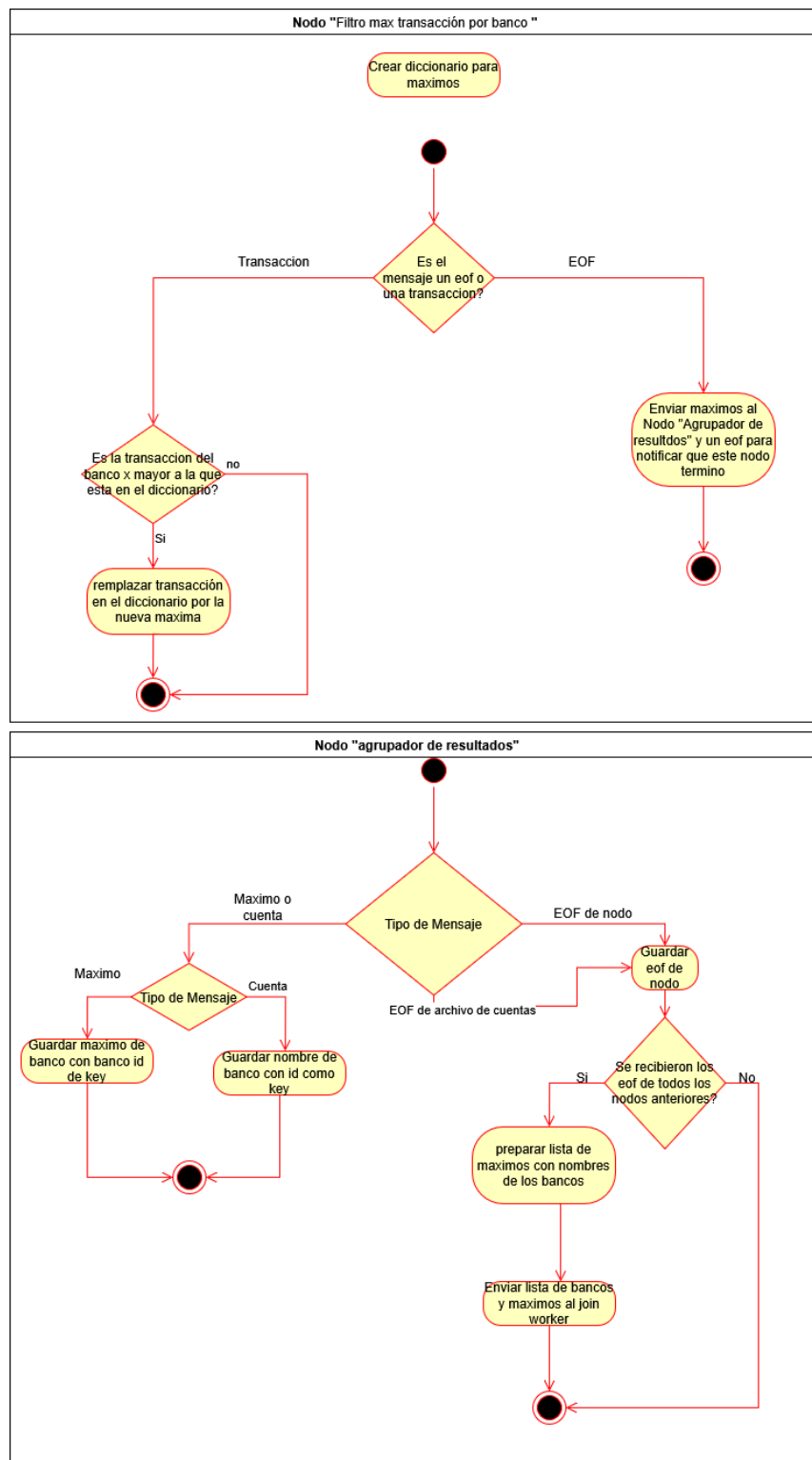




A medida que el Gateway recibe información correspondiente al dataset de transacciones, se la envía a las diferentes instancias del filtro de *Transacciones en USD* mediante un exchange. Este filtro tiene como objetivo descartar todas aquellas transacciones que no se hayan realizado en dólares. Luego, estas transacciones filtradas se envían mediante un exchange hacia el filtro de *Transacciones menores a 50 USD* a medida que se van procesando las transacciones, y en este siguiente filtro se descartarán las transacciones con un monto mayor a 50 USD. Y por último, los resultados de cada uno de los nodos de este filtro se le envían al agrupador de resultados mediante una queue medida que se va realizando el procesamiento.

En cuanto al procesamiento de datos tenemos que los datos procesados y filtrados se le envían al siguiente nodo sobre la marcha. Luego, solo se envía EOF cuando se recibieron tantos EOF como cantidad de nodos del filtro/worker anterior. Es decir, si hay 3 nodos del filtro de *Transacciones en USD* entonces el nodo *Transacciones menores a 50 USD* no enviará EOF al nodo de agrupación de resultados hasta recibir los 3 EOF correspondientes. Para asegurarnos de que todos los nodos reciban todos los EOF correspondientes, este tipo de filtros escalables hacen un broadcast del mensaje de EOF.

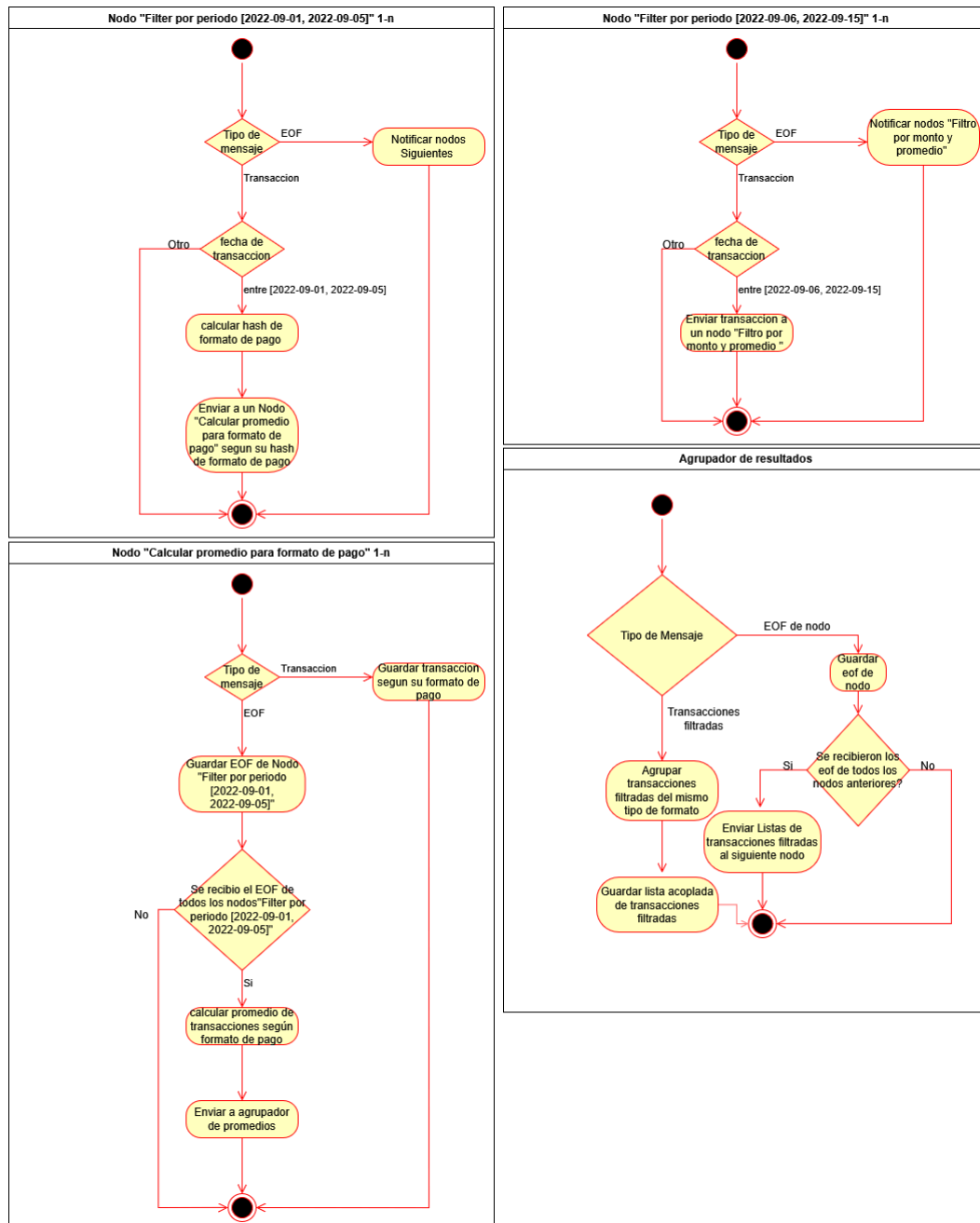
Escenario 2

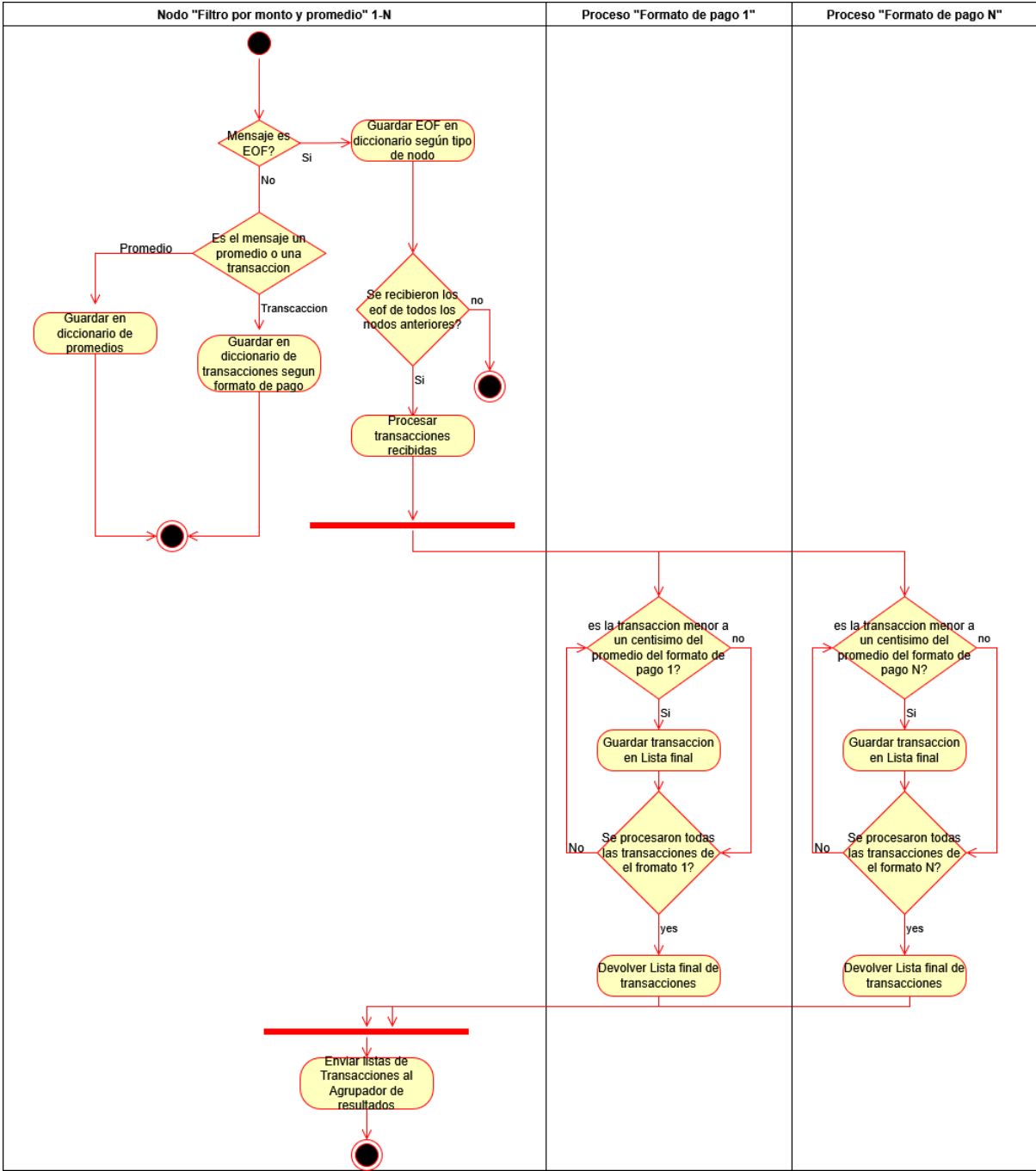


Como se vió en el D.A.G., esta query también utiliza el filtro de *Transacciones en USD*, y el envío de los registros se hace también mediante un exchange, pero en este caso se utiliza como routing key el banco origen de la transacción, de esta manera nos aseguramos que todas las transacciones de cierto banco terminan en el mismo nodo de *Max Transacción por Banco*.

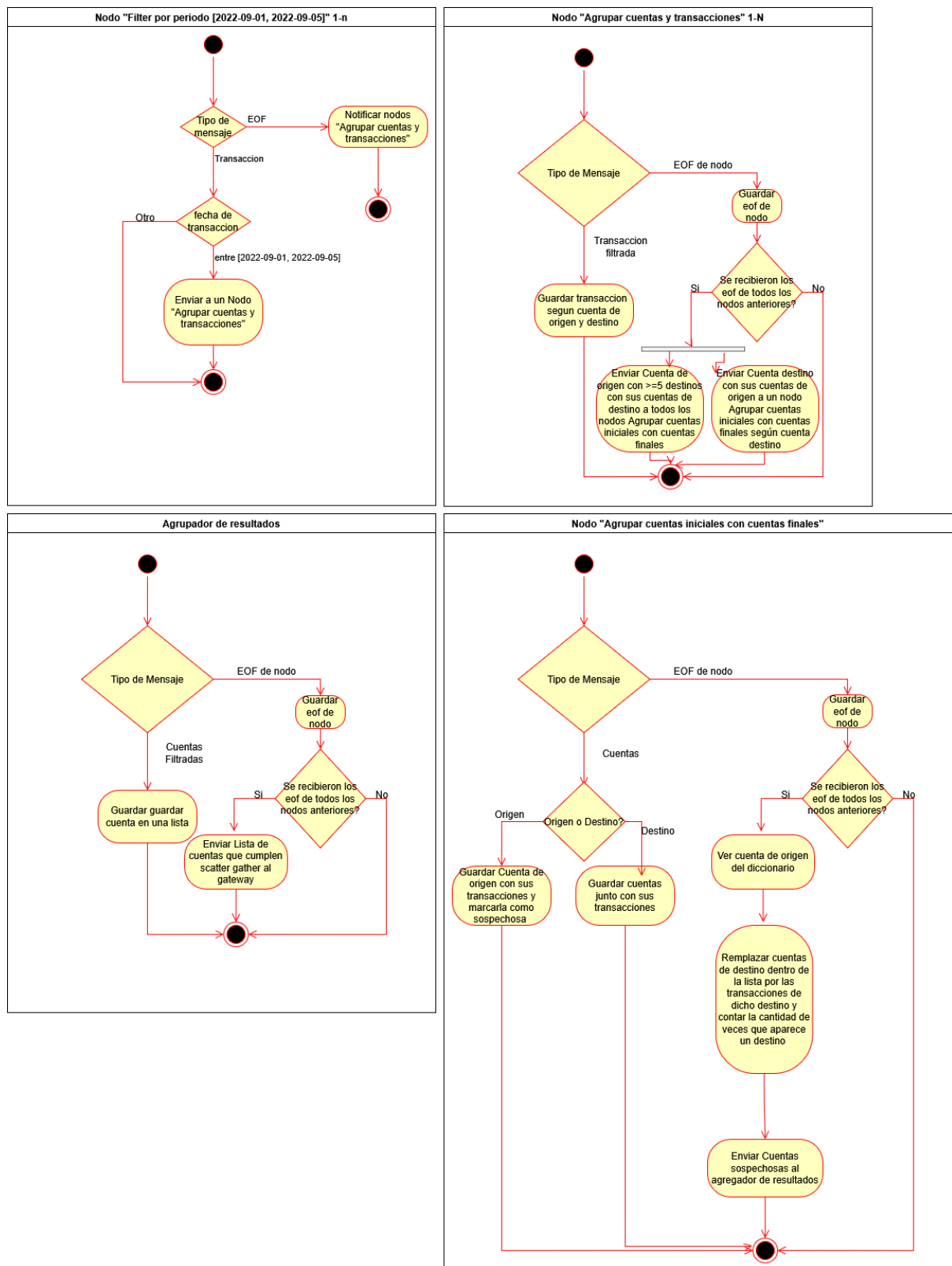
A diferencia del escenario anterior, el filtro de *Max Transacción por Banco* espera a recibir todos los EOF correspondientes antes de enviar los resultados obtenidos, ya que debe esperar a terminar de recibir todos los registros antes de determinar el máximo para luego enviárselo al agregador de resultados.

Escenario 3





Escenario 4



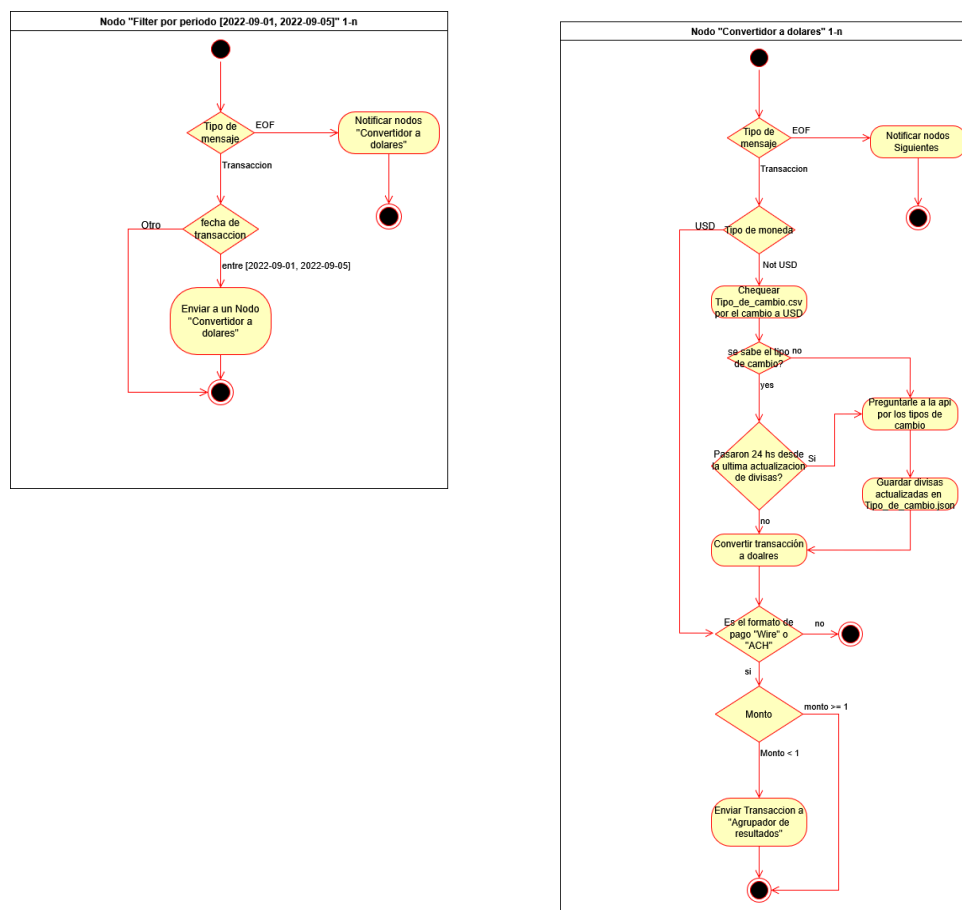
Una vez procesadas las transacciones dentro del filtro por período de tiempo, este le envía a los diferentes nodos de *Agrupar cuentas y transacciones* según el banco y cuenta origen. Luego, estos nodos procesan las transacciones recibidas y se arman dos tipos de diccionarios, uno destinado a

cuentas orígenes con sus respectivas transacciones y otro para cuentas destino también con sus respectivas transacciones. Luego, una vez que se recibieron todos los EOF se le envía a todos los siguientes grupos de nodos (broadcast) aquellos orígenes que cuentas con 5 o más destinos, y las cuentas destino se enviarán a algún nodo siguiente según la cuenta destino.

Después, en el siguiente grupo de nodos se buscan las “cuentas intermedias” y se cuenta la cantidad de veces que estas “cuentas intermedias” realizan una transacción a un mismo destino.

Por último, todos estos resultados son recopilados por un único nodo de *Agrupador de Resultados* y luego se realiza un chequeo final para ver si una cuenta origen le envió a 5 o más “cuentas intermedias” las cuales luego le enviaron a un mismo destino.

Escenario 5



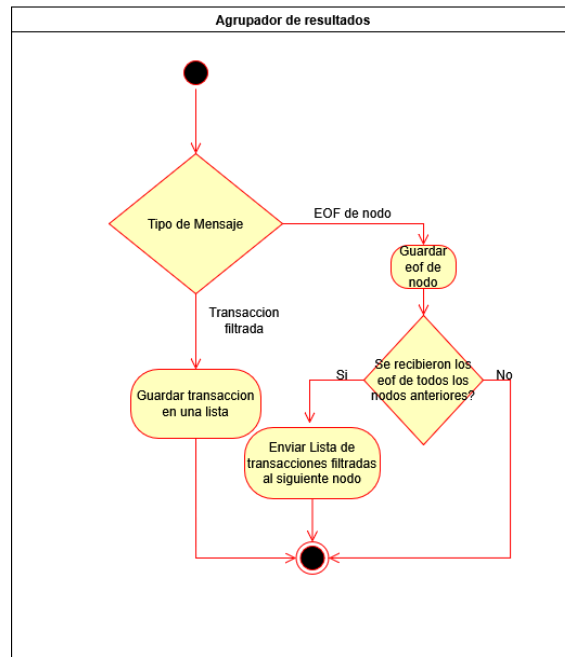
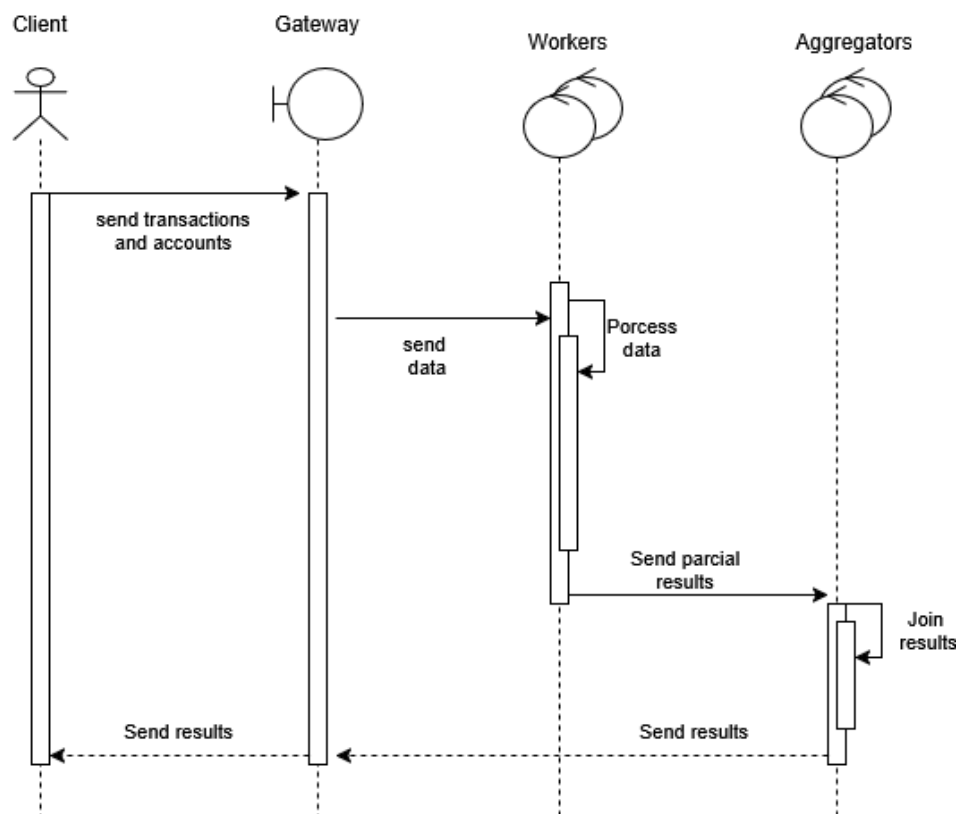


Diagrama de secuencia

La siguiente imagen muestra una vista general de las interacciones entre distintas entidades del sistema a lo largo de la ejecución:



Se logra observar que el Cliente interactúa solamente con el Gateway. Una vez que la conexión con el Gateway sea establecida, el Cliente procederá a enviarle tanto los registros de las transacciones como los de las cuentas. Luego, el Gateway le reenviará estos registros a los Workers correspondientes para realizar las diferentes consultas. Una vez terminado el procesamiento de datos, se le envía al Aggregator correspondiente de la query que se está resolviendo los resultados parciales obtenidos, y luego el Aggregator une todos los resultados recibidos para luego enviárselos al Gateway para que este se los reenvíe al Cliente.

Vista física

Diagrama de robustez

En este caso se definió el posible diseño del sistema teniendo en cuenta la separación de tareas de tal manera que cada caso de uso se mantenga lo más abstraído del otro posible para reducir la propagación de errores del sistema

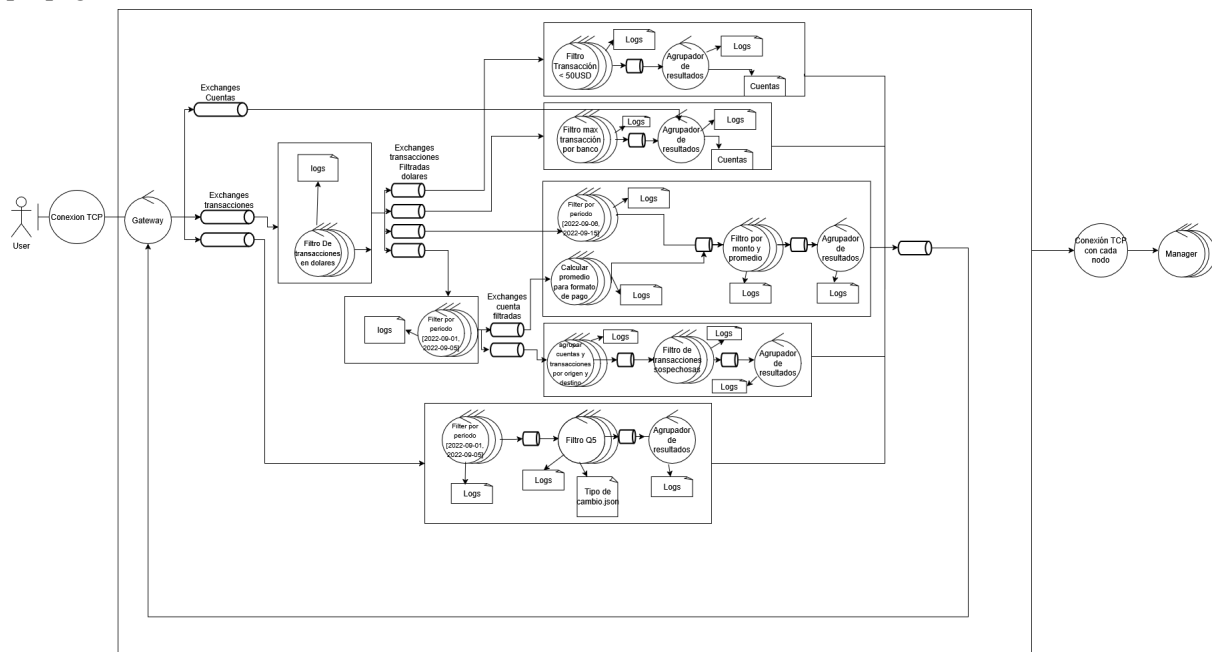
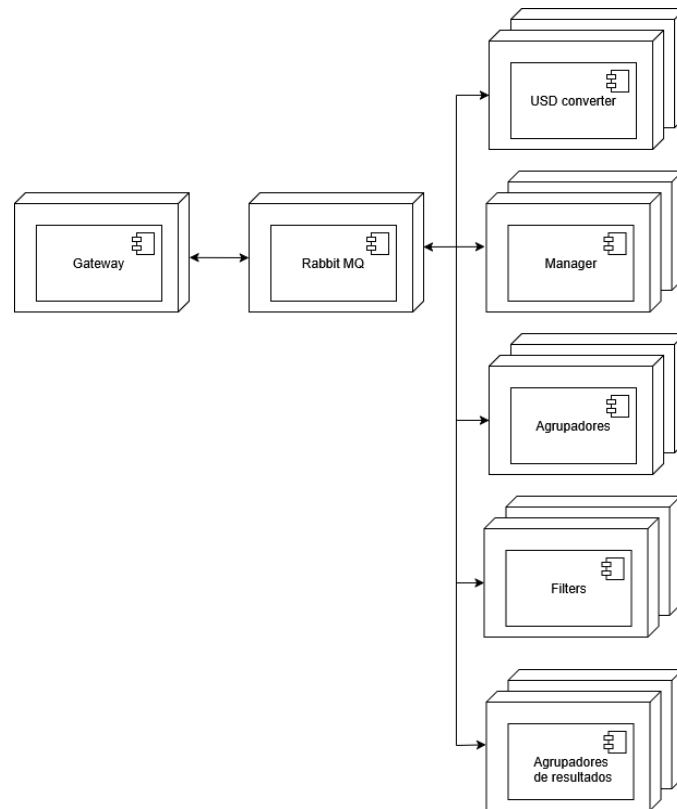


Diagrama de despliegue

En este siguiente diagrama podemos observar las conexiones entre los componentes físicos del sistema. Dentro de las características de este sistema observamos que el cliente es un nodo aparte, el cual se comunica con el nodo Gateway. Luego, los componentes de los filtros, agregadores, convertidores y demás están conectados al componente Rabbit MQ, pues todos los nodos se comunicarán a partir de colas de mensajes.



Tareas a ejecutar

Dentro de este trabajo se detectan las siguientes tareas:

- Middleware:
 - Configuración de la imagen de docker correspondiente al middleware.
 - Implementación middleware
- Protocolo:
 - Definición e implementación del protocolo a utilizar en los diferentes tipos de mensajes como los mensajes de envío de información y mensajes de errores.
- Cliente:
 - Configuración de la imagen de docker correspondiente a los clientes.
 - Envío de registros de transacciones hacia el gateway.
 - Recepción de los resultados.
- Gateway:
 - Implementación de interfaz con el cliente: recepción de registros y envío de resultados.
 - Recepción de resultados del Join.
 - Limpieza de los registros recibidos por el cliente: eliminación de las columnas que no son necesarias para resolver las queries.
 - Envío de los registros recibidos por el cliente hacia el Convertidor de USD.
- Convertidor de USD:
 - Lógica para consultar a la página Frankfurter para saber los valores de conversión.

- Lógica de modificación de los registros para convertir el monto a USD.
- Lógica de lectura y escritura de las queues.
- Filtro de monto:
 - Lógica de filtrado por monto.
 - Lógica de lectura y escritura de las queues.
- Filtro por período de tiempo:
 - Lógica de filtrado por período de tiempo.
 - Lógica de lectura y escritura de las queues.
- Filtro por formato de pago:
 - Lógica de filtrado por formato de pago.
 - Lógica de lectura y escritura de las queues.
- Calculador de promedio:
 - Lógica para calcular el promedio del monto enviado.
 - Lógica de lectura y escritura de las queues.
- Filtro de máxima transacción por banco:
 - Lógica de filtrado por máxima transacción por banco.
 - Lógica de lectura y escritura de las queues.
- Filtro por monto y promedio:
 - Lógica de filtrado por monto y promedio.
 - Lógica de lectura y escritura de las queues.
- Agrupadores:
 - Lógica de agrupación.
 - Lógica de lectura y escritura de las queues.
- Agrupadores de resultados:
 - Lógica de agrupación.
 - Lógica de lectura y escritura de las queues.
- Join:
 - Lógica de agrupación para obtener los resultados finales.
 - Formateo de los resultados.
 - Lógica de lectura y escritura de las queues.

División de los integrantes

A continuación se presenta una división de tareas por cada integrante de manera preliminar:

Tarea	Integrante
Middleware	Germán Gonzalo Escandar Obadía
Protocolo	Micaela J. Alonso Schneider
Cliente	Germán Gonzalo Escandar Obadía
Gateway	Germán Gonzalo Escandar Obadía
Filtros escenarios 1-3	Bruno Gabriel Morseletto
Filtros escenarios 4-5	Micaela J. Alonso Schneider

Agrupadores	Bruno Gabriel Morseletto
Agrupadores de resultados	Bruno Gabriel Morseletto
Join	Micaela J. Alonso Schneider

Decisiones tomadas

Para el escenario número 4 se tomó las siguientes decisiones para el análisis:

- Para que el patrón scatter-gather se cumpla, la cuenta origen debe haberle enviado una transferencia a 5 o más cuentas distintas.
- El notebook provisto por la cátedra para el análisis de resultados fue modificado de la siguiente forma:
 - El merge de los datasets para obtener los pares de cuentas se realizaba haciendo un self join usando el dataset `ranged_trans_usd_sept_df` sin embargo, este dataset solo tenía las cuentas que le enviaban a 5 o más cuentas, así que se perdían las cuentas intermedias que formaban parte del patrón scatter-gather que realizaban transacciones a menos de 5 cuentas. Así que se tomó la decisión de realizar ese merge entre el dataset `ranged_trans_usd_sept_df` y el `trans_usd_sept_1st_df` (el cual tiene todas las transacciones del periodo pedido y cuyas transacciones fueron realizadas en USD).
 - Además, se eliminaron las transacciones duplicadas resultantes del merge entre los dos datasets mencionados anteriormente.

A continuación se observa el código provisto por la cátedra y el código modificado, donde lo marcado en color rojo del lado del código modificado corresponde a las partes alteradas para así lograr realizar el análisis de la interpretación del enunciado y las decisiones tomadas.


```

#4. Accounts that match the scatter-gather pattern and where the source
account has transferred to more than 5 distinct accounts.

accounts_df = ranged_trans_usd_sept_df[["From Bank", "Account", "To Bank",
"Account.1"]]
account_pairs_df = accounts_df.merge(accounts_df, left_on=["To Bank",
"Account.1"], right_on=["From Bank", "Account"]).rename(columns={
    "From Bank_x": "From Bank",
    "Account_x": "From Account",
    "To Bank_y": "To Bank",
    "Account.1_y": "To Account"
})
account_pairs_df = account_pairs_df[(account_pairs_df["From Bank"] !=
account_pairs_df["To Bank"]) | (account_pairs_df["From Account"] !=
account_pairs_df["To Account"])]
account_pairs_df = account_pairs_df.groupby(["From Bank", "From Account", "To
Bank", "To Account"], as_index=False).size()
account_pairs_df = account_pairs_df[(account_pairs_df["size"] > 5)]

from_account_pairs_df = account_pairs_df[["From Bank", "From
Account"]].rename(columns={
    "From Bank": "Bank",
    "From Account": "Account"
})
to_account_pairs_df = account_pairs_df[["To Bank", "To
Account"]].rename(columns={
    "To Bank": "Bank",
    "To Account": "Account"
})
unique_accounts = pd.concat([from_account_pairs_df,
to_account_pairs_df]).drop_duplicates()
unique_accounts

```

Código original correspondiente al escenario número 4.

```

#4. Accounts that match the scatter-gather pattern and where the source
account has transferred to more than 5 distinct accounts.

accounts_df = ranged_trans_usd_sept_df[["From Bank", "Account", "To Bank",
"Account.1"]]
account_pairs_df = accounts_df.merge(trans_usd_sept_1st_df, left_on=["To
Bank", "Account.1"], right_on=["From Bank", "Account"],
how="inner").rename(columns={
    "From Bank_x": "From Bank",
    "Account_x": "From Account",
    "To Bank_y": "To Bank",
    "Account.1_y": "To Account"
})
account_pairs_df = account_pairs_df[(account_pairs_df["From Bank"] !=
account_pairs_df["To Bank"]) | (account_pairs_df["From Account"] !=

```

```

account_pairs_df["To Account"]]]
account_pairs_df = account_pairs_df.drop_duplicates().groupby(["From Bank",
"From Account", "To Bank", "To Account"], as_index=False).size()
account_pairs_df = account_pairs_df[(account_pairs_df["size"] >= 5)]

from_account_pairs_df = account_pairs_df[["From Bank", "From
Account"]].rename(columns={
    "From Bank": "Bank",
    "From Account": "Account"
})
to_account_pairs_df = account_pairs_df[["To Bank", "To
Account"]].rename(columns={
    "To Bank": "Bank",
    "To Account": "Account"
})
unique_accounts = pd.concat([from_account_pairs_df,
to_account_pairs_df]).drop_duplicates()
unique_accounts

```

Código modificado correspondiente al escenario número 4.

Luego, para el escenario número 5 se decidió tomar el valor de conversión para la moneda Bitcoin el valor provisto por la cátedra en el notebook de análisis.

Otra modificación que sufrió el sistema es la eliminación del nodo Join. Pues, este sólo recibía resultados y los enviaba al Gateway, por lo que terminó siendo redundante. Así que se realizó una implementación en donde los Aggregators de cada query le envían sus resultados al Gateway para que luego este le pase los resultados al cliente correspondiente.

Tolerancia a fallos

Node Manager

Los Node Managers tienen el objetivo de escuchar conexiones TCP de los nodos que componen al sistema. Estas conexiones sirven para el envío de heartbeats, los cuales le indican al Node Manager que el nodo en cuestión sigue vivo. En caso de que un nodo no envíe su heartbeat, entonces el Node Manager procederá a reintentar escuchar un heartbeat hasta 3 veces. y en caso de que no se reciba un heartbeat se considerará a ese nodo como caído. Ante la caída de un nodo, el Node Manager líder se encargará de volver a levantarlo. Cabe destacar que, cuando un nodo se apaga de forma ordenada tras recibir la señal *SIGTERM*, este le notifica al Node Manager antes de terminar su ejecución. Por lo tanto, el Node Manager distingue entre un apagado graceful y una caída inesperada por una falla, y no intentará volver a levantar aquellos nodos que se hayan detenido de manera intencional.

En caso de que el Node Manager líder se caiga, se iniciará una nueva elección de líder mediante el Algoritmo de Bully, donde el nuevo líder será aquel con el número de ID mayor.

Una hipótesis y suposición realizada con respecto a los Node Managers es que se considera de que siempre hay al menos un Node Manager levantado, es decir, no se considera el caso de que se caigan todos los Node Managers disponibles.

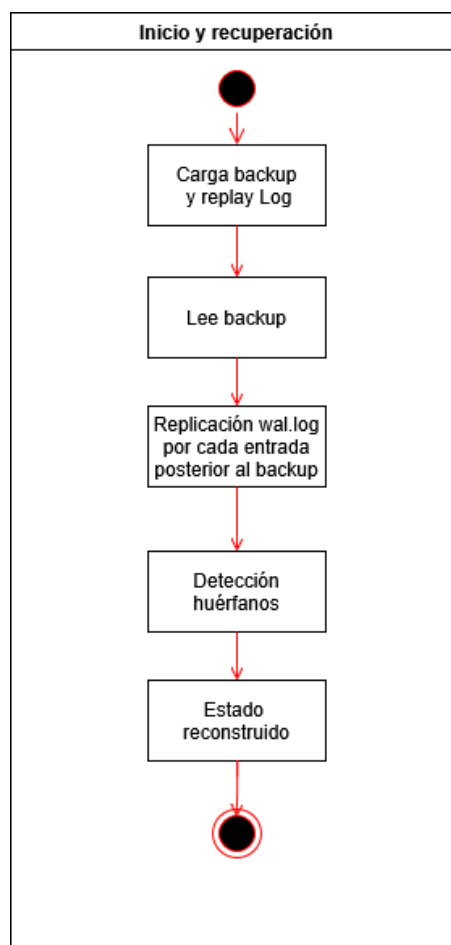
Write-Ahead-Log

El Write-Ahead-Log tiene el objetivo de recuperar el estado de un nodo en caso de que este se caiga y se vuelva a levantar. Este guarda cada mensaje procesado en un archivo log append-only. El append-only nos permite evitar colocar un sistema de bloqueos y sincronización de lectura/escritura en caso de que varios procesos accedan al mismo archivo.

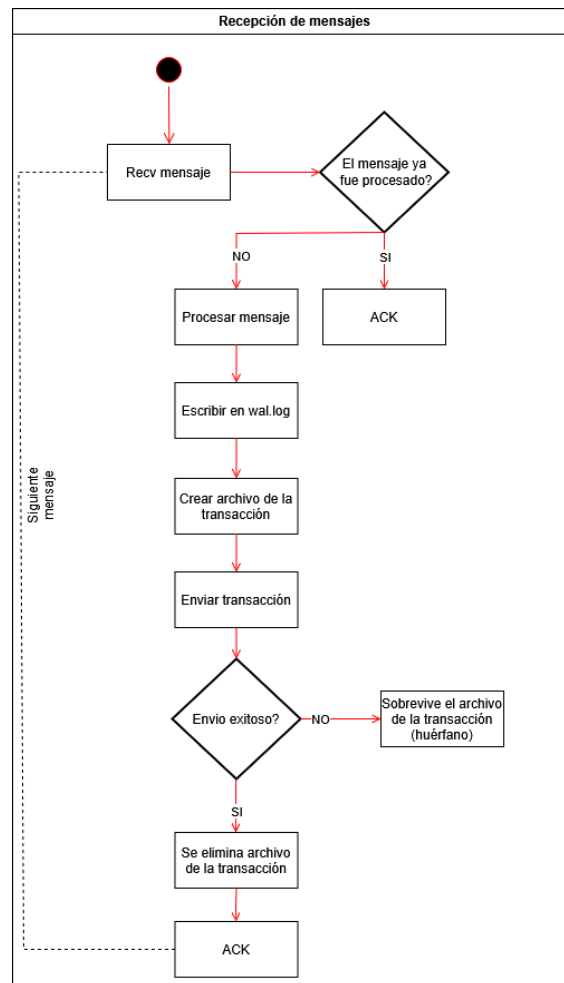
Antes de enviar el resultado del procesamiento de cierto mensaje, se crea un archivo para dicho mensaje, y una vez que se confirma el ACK, se elimina este archivo. Los logs de dicho archivo contienen el número de secuencia, el id y el payload json del mensaje. En caso de que el nodo se caiga entre en el envío y el ACK, entonces el archivo pasa a ser “huérfano” y será considerado al reiniciar el nodo. Al reiniciar, el nodo considerará a estos “huérfanos” como mensajes ya procesados, y de esta forma se evita el re-procesamiento de un mensaje.

Luego, cada 500 líneas se almacena el estado completo en un archivo backup. Este archivo es de utilidad a la hora de reiniciar cierto nodo luego de una caída, pues al reiniciarse, el nodo cargará este archivo backup y verá cuál fue el último mensaje procesado.

Además, se implementó un sistema de Garbage Collector, donde los archivos huérfanos que expiraron sean eliminados.



Flujo de reinicio utilizando Write-Ahead-Log.



Ciclo de procesamiento de mensajes utilizando Write-Ahead-Log.

Chaos Monkey

Se desarrolló una herramienta Chaos Monkey para la simulación de fallas dentro del sistema. A esta herramienta se le puede indicar el intervalo de tiempo (en segundos) entre cada caída y los patrones de contenedores a excluir de las caídas.

Dicha herramienta obtiene los contenedores del sistema y de manera aleatoria agarra uno de los contenedores y lo mata (*kill*). De esta elección aleatoria siempre excluye el contenedor correspondiente a RabbitMQ (pues no se contempla su caída), los contenedores que se indicaron de excluir al levantar la herramienta y se realiza un chequeo de que si la víctima elegida es el último Manager levantado entonces se elige otra víctima.

Gateway

En caso de que el Gateway sw caiga, el Cliente detecta la pérdida de la conexión TCP. Dado que el Cliente no recibió una señal SIGTERM, entonces se interpreta la desconexión como una falla y se reintenta la conexión para así volver a enviar desde cero los datasets de cuentas y transacciones.

Referencias

- [IBM Transactions for Anti Money Laundering \(AML\) - Kaggle](#)
- [Money Laundering Analysis - Kaggle](#)